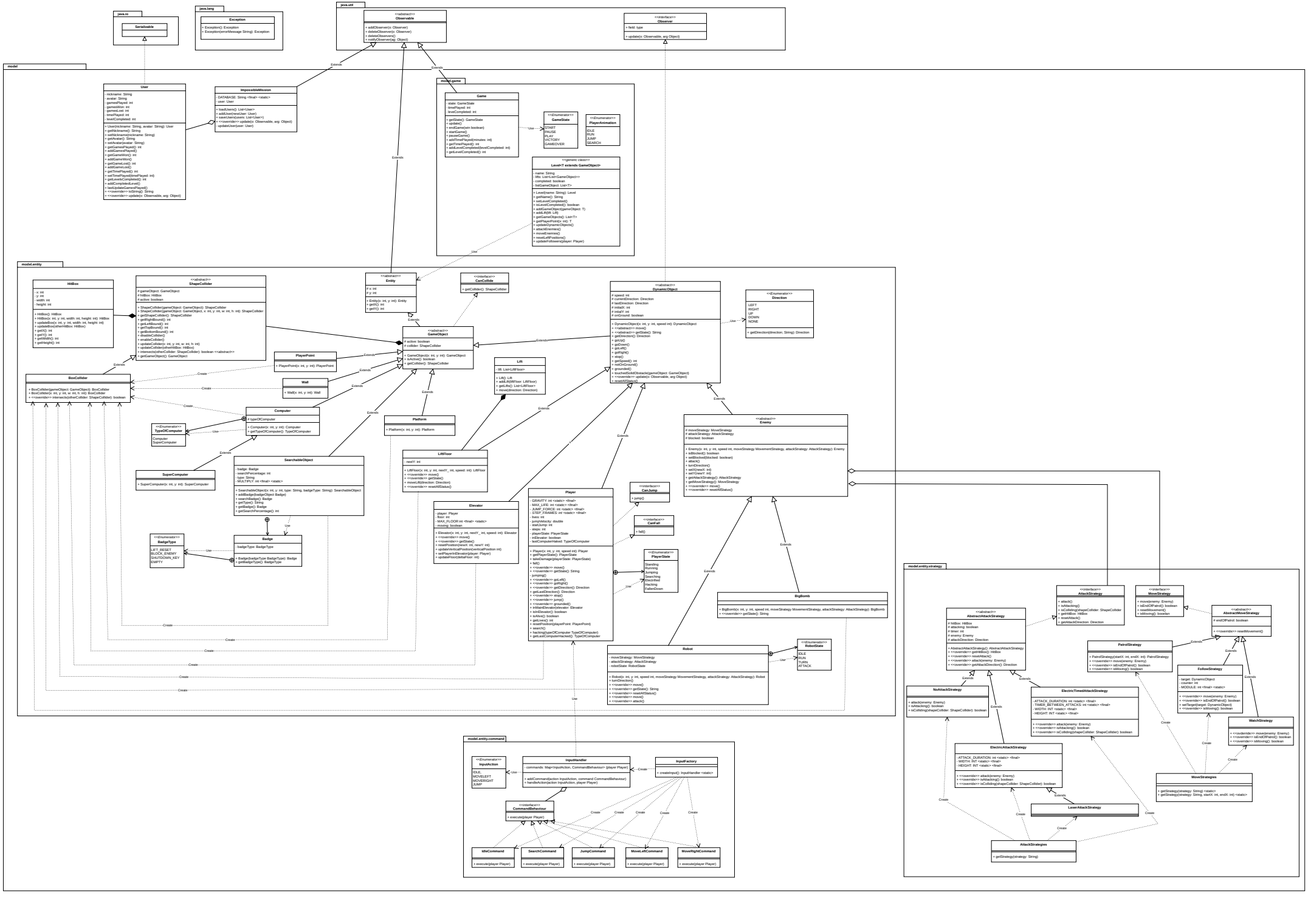

java.io
Serializable
java.lang
Exception
+ Exception(): Exception
+ Exception(errorMessage String): Exception
java.util
<<abstract>>
Observable
+ addObserver(o: Observer)
+ deleteObserver(o: Observer)
+ deleteObservers()
+ notifyObserver(arg: Object)
<<interface>>
Observer
+ field: type
+ update(o: Observable, arg Object)
model
User
- nickname: String
- avatar: String
- gamesPlayed: int
- gamesWon: int
- gamesLost: int
- timePlayed: int
- levelCompleted: int
+ User(nickname: String, avatar: String): User
+ getNickname(): String
+ setNickname(nickname: String)
+ getAvatar(): String
+ setAvatar(avatar: String)
+ getGamesPlayed(): int
+ addGamesPlayed()
+ getGameWon(): int
+ addGameWon()
+ getGameLost(): int
+ addGameLost()
+ getTimePlayed(): int
+ setTimePlayed(timePlayed: int)
+ getLevelCompleted(): int
+ addCompletedLevel()
+ lastUpdateGamesPlayed()
+ <<override>> toString(): String
+ <<override>> update(o: Observable, arg: Object)
ImpossibleMission
- DATABASE: String =final= =static=
- user: User
+ loadUsers(): List<User>
+ addUser(newUser: User)
+ saveUsers(users: List<User>)
+ <<override>> update(o: Observable, arg: Object)
- updateUser(user: User)
model.game
Game
- state: GameState
- timePlayed: int
- levelCompleted: int
+ getState(): GameState
+ update()
+ endGame(win boolean)
+ startGame()
+ pauseGame()
+ addTimePlayed(minutes: int)
+ getTimePlayed(): int
+ addLevelCompleted(levelCompleted: int)
+ getLevelCompleted(): int
<<Enumerator>>
GameState
START
PAUSE
PLAY
VICTORY
GAMEOVER
<<Enumerator>>
PlayerAnimation
IDLE
RUN
JUMP
SEARCH
<<generic class>>
Level<T extends GameObject>
- name: String
- lifts: List<List<GameObject>>
- completed: boolean
- listGameObject: List<T>
+ Level(name: String): Level
+ getName(): String
+ setLevelCompleted()
+ isLevelCompleted(): boolean
+ addGameObject(gameObject: T)
+ addLift(lift: Lift)
+ getGameObjects(): List<T>
+ getPlayerPoint(x: int): T
+ updateDynamicObjects()
+ attackEnemies()
+ moveEnemies()
+ resetLiftPositions()
+ updateFollowers(player: Player)
model.entity
HitBox
- x: int
- y: int
- width: int
- height: int
+ HitBox(): HitBox
+ HitBox(x: int, y: int, width: int, height: int): HitBox
+ updateBox(x: int, y: int, width: int, height: int)
+ updateBox(otherHitBox: HitBox)
+ getX(): int
+ getY(): int
+ getWidth(): int
+ getHeight(): int
<<abstract>>
ShapeCollider
gameObject: GameObject
hitBox: HitBox
active: boolean
+ ShapeCollider(gameObject: GameObject): ShapeCollider
+ ShapeCollider(gameObject: GameObject, x: int, y: int, w: int, h: int): ShapeCollider
+ getShapeCollider(): ShapeCollider
+ getRightBound(): int
+ getLeftBound(): int
+ getTopBound(): int
+ getBottomBound(): int
+ disableCollider()
+ enableCollider()
+ updateCollider(x: int, y: int, w: int, h: int)
+ updateCollider(otherHitBox: HitBox)
+ intersects(otherCollider: ShapeCollider): boolean <<abstract>>
+ getGameObject(): GameObject
<<abstract>>
Entity
x: int
y: int
+ Entity(x: int, y: int): Entity
+ getX(): int
+ getY(): int
<<interface>>
CanCollide
+ getCollider(): ShapeCollider
<<abstract>>
GameObject
active: boolean
collider: ShapeCollider
+ GameObject(x: int, y: int): GameObject
+ isActive(): boolean
+ getCollider(): ShapeCollider
<<abstract>>
DynamicObject
speed: int
currentDirection: Direction
lastDirection: Direction
initialX: int
initialY: int
onGround: boolean
+ DynamicObject(x: int, y: int, speed int): DynamicObject
+ <<abstract>> move()
+ <<abstract>> getState(): String
+ getDirection(): Direction
+ goUp()
+ goDown()
+ goLeft()
+ goRight()
+ stop()
+ getSpeed(): int
+ notOnGround()
+ grounded()
+ touchesSolidObstacle(gameObject: GameObject)
+ <<override>> update(o: Observable, arg Object)
+ resetAllStates()
<<Enumerator>>
Direction
LEFT
RIGHT
UP
DOWN
NONE
+ getDirection(direction: String): Direction
PlayerPoint
+ PlayerPoint(x: int, y: int): PlayerPoint
Wall
+ Wall(x: int, y: int): Wall
Lift
- lift: List<LiftFloor>
+ Lift(): Lift
+ addLift(liftFloor: LiftFloor)
+ getLifts(): List<LiftFloor>
+ move(direction: Direction)
BoxCollider
+ BoxCollider(gameObject: GameObject): BoxCollider
+ BoxCollider(x: int, y: int, w: int, h: int): BoxCollider
+ <<override>> intersects(otherCollider: ShapeCollider): boolean
Computer
typeOfComputer
+ Computer(x: int, y: int): Computer
+ getTypeOfComputer(): TypeOfComputer
<<Enumerator>>
TypeOfComputer
Computer
SuperComputer
Platform
+ Platform(x: int, y: int): Platform
LiftFloor
- nextY: int
+ LiftFloor(x: int, y: int, nextY_: int, speed: int): LiftFloor
+ <<override>> move()
+ <<override>> getState()
+ moveLift(direction: Direction)
+ <<override>> resetAllStatus()
<<abstract>>
Enemy
moveStrategy: MoveStrategy
attackStrategy: AttackStrategy
blocked: boolean
+ Enemy(x: int, y: int, speed int, moveStrategy MovementStrategy, attackStrategy: AttackStrategy): Enemy
+ isBlocked(): boolean
+ setBlocked(blocked: boolean)
+ attack()
+ turnDirection()
+ setX(newX: int)
+ setY(newY: int)
+ getAttackStrategy(): AttackStrategy
+ getMoveStrategy(): MoveStrategy
+ <<override>> move()
+ <<override>> resetAllStatus()
SearchableObject
- badge: Badge
- searchPercentage: int
- type: String
- MULTIPLY: int =final= =static=
+ SearchableObject(x: int, y: int, type: String, badgeType: String): SearchableObject
+ addBadge(badgeObject Badge)
+ setBadge(): Badge
+ getType(): String
+ getBadge(): Badge
+ getSearchPercentage(): int
SuperComputer
+ SuperComputer(x: int, y: int): SuperComputer
Badge
- badgeType: BadgeType
+ Badge(badgeType BadgeType): Badge
+ getBadgeType(): BadgeType
<<Enumerator>>
BadgeType
LIFT_RESET
BLOCK_ENEMY
SHUTDOWN_KEY
EMPTY
Elevator
- player: Player
- floor: int
- MAX_FLOOR int =final= =static=
- moving: boolean
+ Elevator(x: int, y: int, nextY_ int, speed: int): Elevator
+ <<override>> move()
+ <<override>> getState()
+ moveElevator(x: int, newY: int)
+ updateVerticalPosition(verticalPosition int)
+ setPlayerInElevator(player: Player)
+ updateFloor(defaultFloor: int)
Player
- GRAVITY: int =static= =final=
- MAX_LIFE: int =static= =final=
- JUMP_FORCE: int =static= =final=
- STEP_FRAMES: int =static= =final=
- lives: int
- jumpVelocity: double
- startJump: int
- steps: int
- playerState: PlayerState
- inElevator: boolean
- lastComputerHacked: TypeOfComputer
+ Player(x: int, y: int, speed int): Player
+ getPlayerState(): PlayerState
+ takeDamage(playerState: PlayerState)
+ fall()
+ <<override>> move()
+ <<override>> getState(): String
- jumping()
+ <<override>> goLeft()
+ <<override>> goRight()
+ <<override>> getDirection(): Direction
+ getLastDirection(): Direction
+ <<override>> stop()
+ <<override>> jump()
+ <<override>> grounded()
+ inMainElevatorArea(elevator: Elevator
+ isInElevator(): boolean
+ isAlive(): boolean
+ getLives(): int
+ resetPosition(playerPoint: PlayerPoint)
+ search()
+ hacking(typeOfComputer TypeOfComputer)
+ getLastComputerHacked(): TypeOfComputer
<<interface>>
CanJump
+ jump()
<<interface>>
CanFall
+ fall()
<<Enumerator>>
PlayerState
Standing
Running
Jumping
Searching
Electrified
Hacking
FallenDown
Robot
- moveStrategy: MoveStrategy
- attackStrategy: AttackStrategy
- robotState: RobotState
+ Robot(x: int, y: int, speed int, moveStrategy MovementStrategy, attackStrategy: AttackStrategy): Robot
+ turnDirection()
+ <<override>> move()
+ <<override>> getState(): String
+ <<override>> resetAllStatus()
+ <<override>> move()
+ <<override>> attack()
<<Enumerator>>
RoboState
IDLE
RUN
TURN
ATTACK
BigBomb
+ BigBomb(x: int, y: int, speed int, moveStrategy MovementStrategy, attackStrategy: AttackStrategy): BigBomb
+ <<override>> getState(): String
model.entity.command
<<Enumerator>>
InputAction
IDLE
MOVELEFT
MOVERIGHT
JUMP
InputHandler
- commands: Map<InputAction, CommandBehaviour> (player Player)
+ addCommand(action InputAction, command CommandBehaviour)
+ handleAction(action InputAction, player Player)
InputFactory
+ create(input): InputHandler <static>
<<interface>>
CommandBehaviour
+ execute(player Player)
IdleCommand
+ execute(player Player)
SearchCommand
+ execute(player Player)
JumpCommand
+ execute(player Player)
MoveLeftCommand
+ execute(player Player)
MoveRightCommand
+ execute(player Player)
model.entity.strategy
<<interface>>
AttackStrategy
+ attack()
+ isAttacking()
+ isColliding(shapeCollider: ShapeCollider
+ getHitBox: HitBox
+ resetAttack()
+ getAttackDirection: Direction
<<interface>>
MoveStrategy
+ move(enemy: Enemy)
+ isEndOfPatrol(): boolean
+ resetMovement()
+ isMoving(): boolean
<<abstract>>
AbstractAttackStrategy
hitBox: HitBox
attacking: boolean
timer: int
enemy: Enemy
attackDirection: Direction
+ AbstractAttackStrategy(): AbstractAttackStrategy
+ <<override>> getHitBox(): HitBox
+ <<override>> resetAttack()
+ <<override>> attack(enemy: Enemy)
+ <<override>> getAttackDirection(): Direction
<<abstract>>
AbstractMoveStrategy
endOfPatrol: boolean
+ <<override>> resetMovement()
PatrolStrategy
+ PatrolStrategy(startX: int, endX: int): PatrolStrategy
+ <<override>> move(enemy: Enemy)
+ <<override>> isEndOfPatrol(): boolean
+ <<override>> isMoving(): boolean
FollowStrategy
- target: DynamicObject
- counter: int
- MODULE: int =final= =static=
+ <<override>> move(enemy: Enemy)
+ <<override>> isEndOfPatrol(): boolean
+ setTarget(target: DynamicObject)
+ <<override>> isMoving(): boolean
WatchStrategy
+ <<override>> move(enemy: Enemy)
+ <<override>> isEndOfPatrol(): boolean
+ <<override>> isMoving(): boolean
NoAttackStrategy
+ attack(enemy: Enemy)
+ isAttacking(): boolean
+ isColliding(shapeCollider: ShapeCollider): boolean
ElectricTimedAttackStrategy
- ATTACK_DURATION: int =static= =final=
- TIMER_BETWEEN_ATTACKS: int =static= =final=
- WIDTH: INT =static= =final=
- HEIGHT: INT =static= =final=
+ <<override>> attack(enemy: Enemy)
+ <<override>> isAttacking(): boolean
+ <<override>> isColliding(shapeCollider: ShapeCollider): boolean
ElectricAttackStrategy
- ATTACK_DURATION: int =static= =final=
- WIDTH: INT =static= =final=
- HEIGHT: INT =static= =final=
+ <<override>> attack(enemy: Enemy)
+ <<override>> isAttacking(): boolean
+ <<override>> isColliding(shapeCollider: ShapeCollider): boolean
MoveStrategies
+ getStrategy(strategy: String) <static>
+ getStrategy(strategy: String, startX: int, endX: int) <static>
LaserAttackStrategy
AttackStrategies
+ getStrategy(strategy: String)
Extends
Create
Use